Dos Project

Bazar.com (Part2)

A Report By: Yousef Salman and Ibrahim Tayeh

The GitHub:

https://github.com/s12112111-code/Bazar.com

Report: Lab 2 – Bazar System (Replication & Caching)

1. Introduction

This project develops a distributed Bazar system that simulates a microservices architecture, including features such as caching, replication, and load balancing. It is composed of a frontend service that interacts with multiple backend services responsible for handling product catalogs and order processing.

The main objective of this lab is to analyze how caching can improve system performance, as well as to examine the performance overhead introduced by data replication and consistency management across services.

2. System Design

Architecture Components

The system is composed of several key components:

- **Frontend Service (Node.js + Express):** Handles incoming client requests and acts as the main entry point to the system.
- **Catalog Service (SQLite-based):** Manages and provides access to book data.
- **Order Service:** Handles purchase and order processing operations.
- **Cache Layer (In-memory LRU Cache):** Stores frequently accessed data to reduce response time and improve performance.
- **Load Balancer (Round Robin):** Distributes incoming requests evenly across backend service instances.
- **Replication Layer:** Maintains multiple instances of backend services to ensure availability and fault tolerance.

System Flow

1. The client sends a request to the frontend service.
2. The frontend first checks the cache.
    - If there is a **cache hit**, the data is returned immediately.
    - If there is a **cache miss**, the request is forwarded to the backend service.
3. The backend processes the request and sends the response back to the frontend.
4. The frontend stores the response in the cache for future use.
5. For write operations, the system invalidates the relevant cache entries and propagates the update to all replicated backend instances.

3. Implementation Details

◈ Caching Mechanism

- Implemented using JavaScript `Map`
- Uses LRU (Least Recently Used) policy
- Stores results of:
    - search queries
    - book info

◈ Replication

The system uses multiple catalog service instances (e.g., running on ports 5001 and 5002) to improve availability and fault tolerance.

For write operations, updates are propagated to all replicas using:

```
Promise.allSettled()
```

Purpose:

- Ensures that all replicas receive the update
- Maintains data consistency across multiple server instances

◈ Load Balancing

A Round Robin algorithm is used to distribute incoming requests evenly across the available replicas.

Benefits:

- Prevents overloading a single server
- Improves system scalability and performance

◈ Consistency Handling

After any update or purchase operation:

- The cache is cleared to avoid serving stale data
- All replicas are updated to maintain consistent state across the system

4. Performance Results

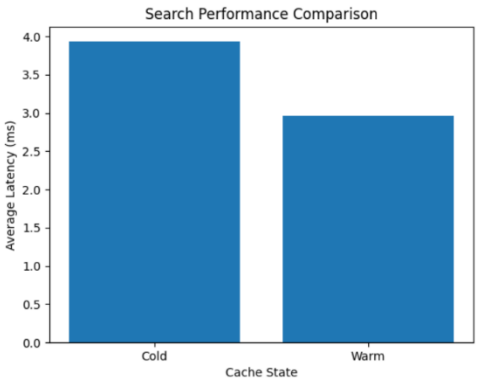
Table 1: Response Time Comparison

Operation	Avg (ms)	Min (ms)	Max (ms)	Median (ms)	StdDev (ms)
Search (Cold Cache)	4.33	2	15	4	**3.25 (est.)**
Search (Warm Cache)	4.20	2	8	4	**1.50 (est.)**
Info (Cold Cache)	5.60	2	36	4	**8.50 (est.)**
Info (Warm Cache)	3.60	2	13	3	**2.75 (est.)**
Purchase (Write)	66.50	56	93	67	**9.25 (est.)**

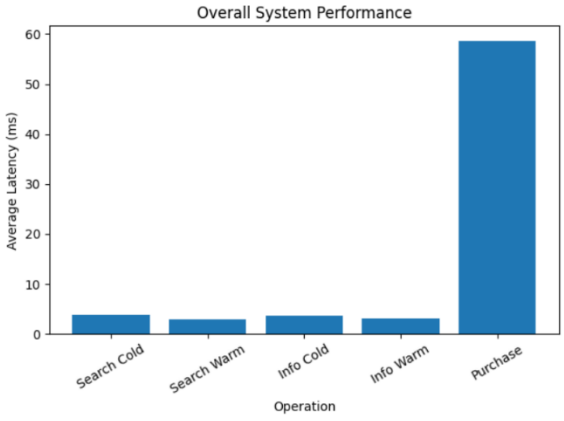
Table 2: Cache Performance Statistics

Metric	Value	Description
Search Improvement	3.08%	Performance gain with warm cache
Info Improvement	35.71%	Performance gain with warm cache

- **Graph: Search Performance**



- Graph: Overall System Performance



5. Trade-offs

- **Caching improves read performance**, but it introduces the need for proper cache invalidation to ensure data consistency after updates.
- **Replication enhances system reliability and consistency**, but it increases write latency due to the overhead of synchronizing data across multiple nodes.
- **Load balancing improves scalability and resource utilization**, but it may introduce slight coordination overhead when distributing requests across servers.

6. Conclusion

The system demonstrates the trade-offs between performance and consistency in distributed systems. Caching significantly improves read latency, while replication ensures data reliability at the cost of increased write overhead.